

PROOF OF CONCEPT

Native PWA Experiment

A synthetic case showing that a progressive web app today can be indistinguishable from a native mobile or desktop application — including the UI itself.

Installable

Offline

Per-platform native UI



whatpwacando.today catalogs what the web *can* do. This experiment shows how it can **feel**.



One synthetic app

"Streaks", a daily habit tracker — small enough to read, real enough to judge.



Native UI per platform

Cupertino on iOS & macOS, Material on Android & Linux, Fluent on Windows.



Judged like a native app

Install flow, offline start, dark mode, safe areas, platform widgets and gestures.

1

codebase — domain logic shared by every UI

4

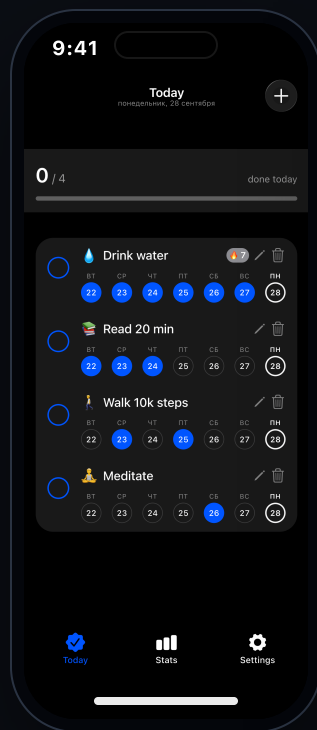
native design systems from one tree

0

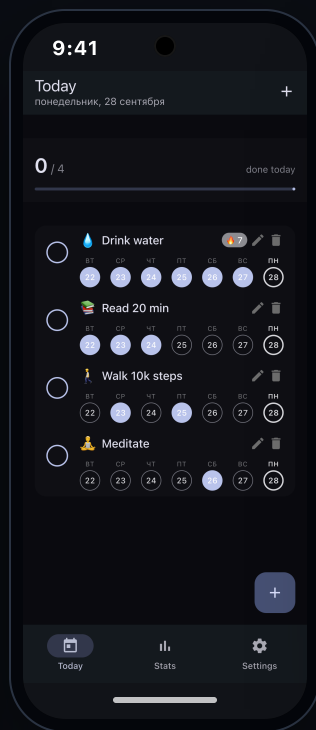
app store reviews to ship an update

Streaks — the same app, six design languages

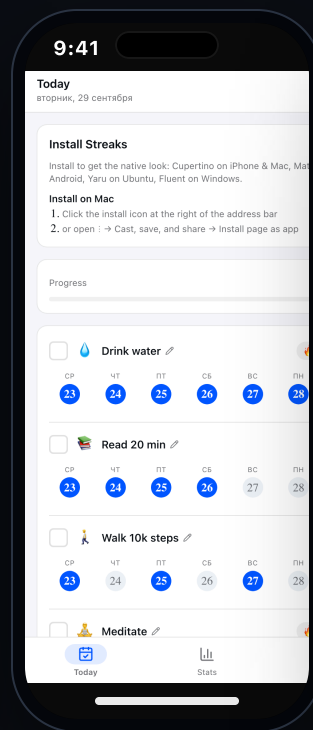
The design language is chosen at runtime — detected for installed PWAs, forceable via `?design=...`



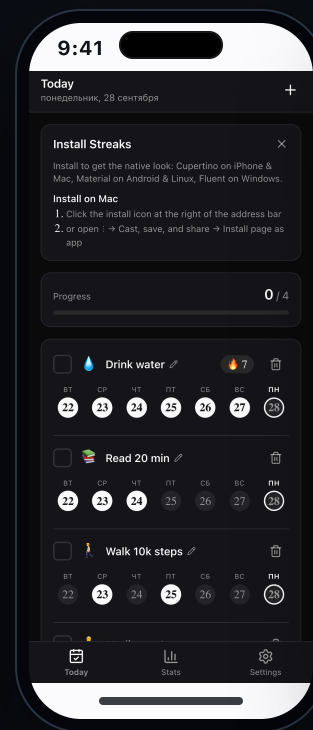
Cupertino · iOS / macOS



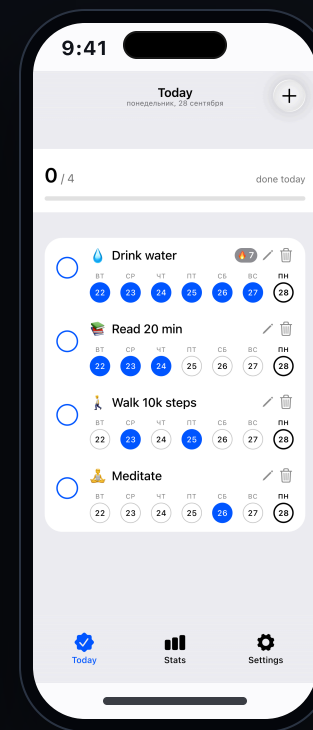
Material · Android / Linux



custom · browser tab



shadcn/ui · neutral

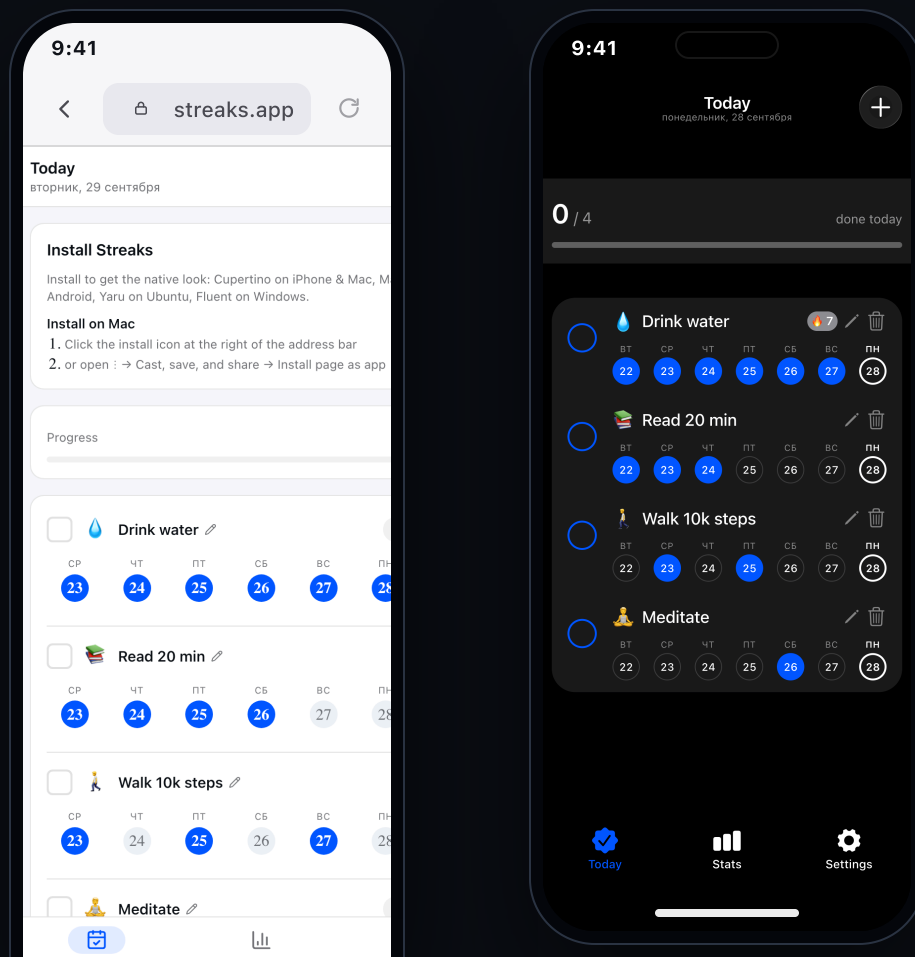


Light & dark · every kit

SAME URL, TWO EXPERIENCES

Open the link — or install it

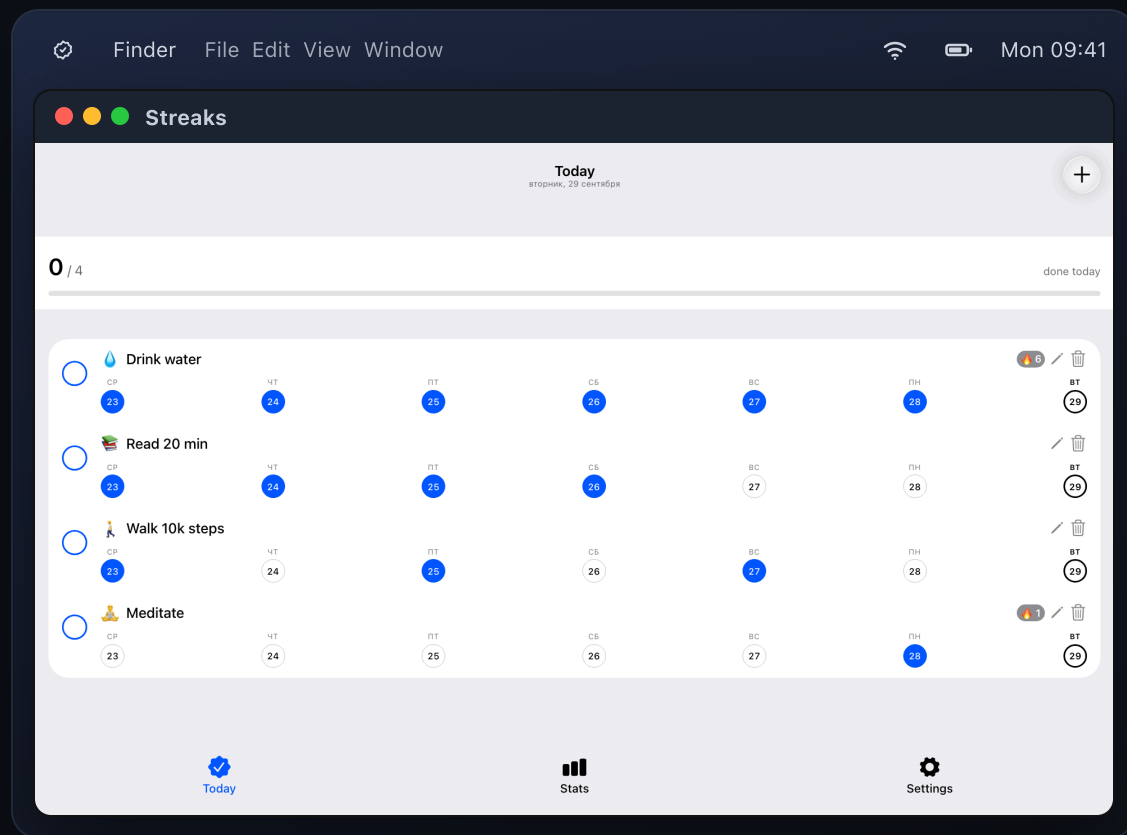
One build, one URL. A browser tab shows the app's own design, chrome and all; install it and the UI is handed over to your platform.



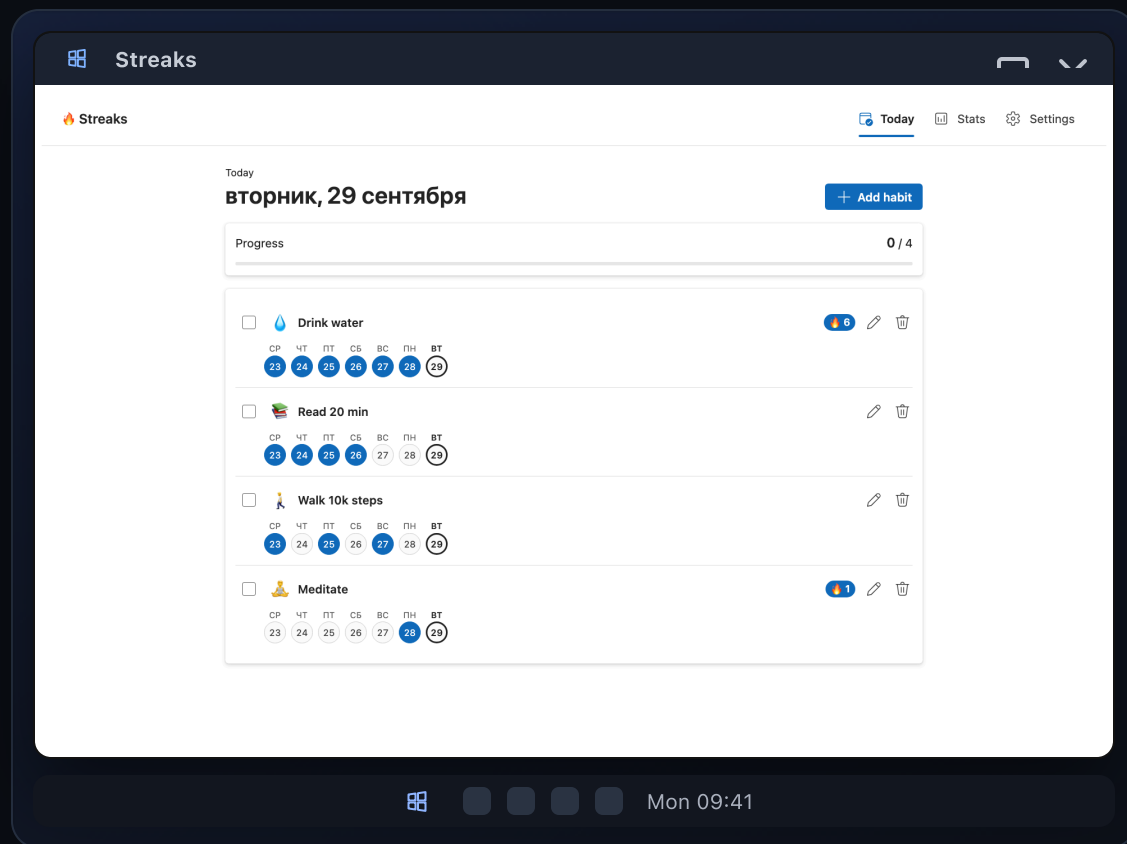
DESKTOP, TOO

The same app as a desktop app

Install it on macOS, Windows or Linux and it opens in its own window, drawn in that platform's design language.



macOS · Cupertino



Windows · Fluent

One domain layer, swappable UI kits

Domain — src/domain

Pure reducer, versioned localStorage persistence, streak & progress selectors — TypeScript, zero UI imports, unit-tested.

Platform — src/platform

Detects OS, browser, display mode and install status; resolves the design language (override → installed → custom) and owns the install prompt store.

PWA shell

vite-plugin-pwa: manifest, auto-updating service worker, per-kit lazy chunks, offline precache.



Cupertino · Material

Framework7 9 — one shell, two native languages:

`cupertino` draws the Apple look, `material` the Android one. F7 icons, swipe actions, FAB, system dialogs.

Fluent

@fluentui/react-components v9 — real Fluent widgets, CSS-in-JS design tokens, TabList navigation, dark theme from the OS.

Yaru · custom · shadcn

Own Flutter-style widget kit — inline styles from a single theme file (`theme.jsx`); three languages, no CSS frameworks.

IT BEHAVES LIKE AN APP, NOT A PAGE

Native functionality you can demo today



Installs like an app

Own icon on the home screen or dock, opens in a standalone window — no browser chrome, no store.



Works offline

Service worker precaches the whole build — flip on airplane mode and cold-start it.



Platform widgets & gestures

FAB and pill tabs on Android, circular checks and swipe-to-delete on iOS, TabList on Windows, a headerbar on Ubuntu.



System-feel interactions

Tap-to-complete, per-day toggling, native-style dialogs and confirmations, confetti on a perfect day.



Follows the system

Dark and light themes, safe-area insets for notches and home bars, locale-aware dates.



Zero-friction updates

The auto-updating service worker ships the next version on the next launch — no review queue.

PWA vs native binary — what's left on the table?

	THIS EXPERIMENT (PWA)	NATIVE BINARY
Distribution	A URL — no store, no review	App store submission & review
Updates	Instant, on next load (autoUpdate SW)	Release cycle + user update
UI quality	Platform UI kits per OS	Platform UI kits
Offline	Precached app shell + data	Full offline
Deep OS access	Moderate — web APIs only	Full platform access
Codebase	One for every platform	Per-platform (Swift/Kotlin/...)

Honest scope: a PWA is not a replacement for apps that need deep OS integration — it is a serious answer for everything else.

Not the first to bet on the web



What PWA Can Do Today

The capability catalog this experiment pairs with: what browsers support, per platform — whatpwacando.today



Excalidraw · Squoosh

Flagship installable PWAs — app-grade UX fully in the browser, offline-capable.



Starbucks · Uber · X Lite

Production PWAs at scale — lighter, faster to market, near-parity with their native apps.



Framework7 · Fluent UI

The kits that make platform-native looks possible from web code.



web.dev/learn/pwa · PWA Stats

Reference learning path and outcome statistics from the field.

Run it in a minute

```
git clone
https://github.com/nikita-fedorov008/NativePWAExperiment
cd NativePWAExperiment
npm install
npm run dev    # http://localhost:5173
npm run build  # static dist/ - host anywhere
npm test      # unit tests
npm run typecheck # tsc --noEmit, strict
```

Open it in a browser tab for the custom look — then install it and watch the UI switch to your platform's design language.

```
src/
├─ domain/ habits · celebration – pure logic + tests
├─ platform/ detection · install store
├─ app/ composition root
└─ packages/ui-kit/ – workspace package
    ├─ designSystems.js – the six languages
    ├─ theme.jsx + widgets.jsx – code kit
    └─ kits/ f7 · fluent · code
```

TRY IT YOURSELF

Judge it on your own phone.



Open the URL

in Safari or Chrome



Tap Install

from the banner or browser menu



Launch it

the native UI of your platform greets you

github.com/nikita-fedorov008/NativePWAExperiment